Andrês Oliveira

LaTeX para gente grande

LEM

Andrês Oliveira

LaTeX para gente grande

Um guia avançado

2025

Ano 2025

MIT License

Copyright (c) 2025 Andrês

This book is licensed under the MIT License.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Conteúdo

Introdução 1

Escopo do minicurso 1

Relação entre Livro e Slides 2

O que é LaTeX? 3

Estrutura de Projeto Profissional 3

Compiladores, Fontes e Automação 5

Ecossistema: Distribuição, Editor, Compilador, Classe e Pacote 5

Fluxo de Compilação e Arquivos Auxiliares 6

Por que múltiplas compilações? 7

Anatomia dos Arquivos Gerados 9

Compiladores: pdflatex, xelatex e lualatex 10

Fontes no pdfLaTeX 10

Fontes do sistema e arquivos de fonte 11

Microtipografia com microtype 13

Automação e Fluxos de Trabalho 15

Erros e Diagnóstico: Lendo o Log 17

Consulta de Documentação 19

Documentos técnicos e escrita matemática 23

Estrutura de um documento técnico 24

A pilha AMS 25

Ambientes de equações 26

Numeração e referências 28

Texto dentro de matemática 29

Operadores matemáticos 31

Macros semânticas 32

Definição de comandos 33

Cabeçalhos e rodapés 35

Delimitadores e notação pareada 36

Matrizes, casos e pontos 39

Um exemplo de refatoração 40

Critérios práticos 41

I
Introdução

Este livro constitui o material de referência para o minicurso "LaTeX para gente grande", realizado no Laboratório de Ensino de Matemática (LEM) do Instituto de Matemática, Estatística e Computação Científica (IMECC) da Universidade Estadual de Campinas (UNICAMP). O conteúdo foi desenvolvido para usuários que já possuem familiaridade básica com o sistema LaTeX e precisam produzir documentos técnicos com maior controle sobre estrutura, compilação, fontes, matemática e figuras.

I.1 Escopo do minicurso

O curso trata o LaTeX como um sistema de preparação de documentos. A ênfase não está na memorização de comandos isolados, mas na compreensão das relações entre arquivo-fonte, compilador, pacotes, arquivos auxiliares e documento final. Essa perspectiva permite diagnosticar erros, organizar projetos maiores e tomar decisões técnicas de forma explícita.

O material privilegia os seguintes temas:

- **Fluxos de trabalho reprodutíveis**: uso de automação e ferramentas de build para reduzir dependências implícitas no processo de compilação.
- **Organização modular**: Estruturação de projetos grandes em múltiplos arquivos, facilitando a manutenção e a colaboração via sistemas de controle de versão.
- **Tipografia**: configuração de codificação, fontes e microtipografia no fluxo com `pdfLaTeX`.
- **Matemática e diagramas**: uso de pacotes da AMS, `mathtools` e TikZ para representação técnica.
- **Apresentações**: criação de slides em Beamer, com atenção à estrutura, ao ritmo de aula e aos exemplos.

I.2 Relação entre Livro e Slides

Este livro funciona como uma apostila densa e autossuficiente. Cada capítulo desenvolve os temas com explicações, justificativas técnicas e, quando necessário, referências aos manuais oficiais. A leitura do livro deve permitir que o aluno retome o conteúdo depois da aula sem depender dos slides.

Os slides, por outro lado, servem como roteiro para as atividades presenciais de uma tarde de aula. Eles contêm sínteses, demonstra-

ções práticas e laboratórios guiados. Embora os slides conduzam a dinâmica da aula, é neste texto que o leitor encontrará o suporte conceitual para estudar os problemas com mais detalhe.

1.3 O que é LaTeX?

Historicamente, o sistema TeX foi criado por Donald Knuth no final da década de 1970 com o objetivo de permitir a autores a produção de livros com alta qualidade tipográfica, especialmente em matemática. O LaTeX, desenvolvido por Leslie Lamport na década de 1980, surgiu como um conjunto de macros que abstrai a complexidade do TeX, permitindo que o autor foque na estrutura lógica do documento enquanto o sistema cuida da apresentação visual.

Diferente de editores *WYSIWYG* (*What You See Is What You Get*), o LaTeX adota a filosofia de separação entre conteúdo e forma. O autor escreve um arquivo de texto puro contendo marcações; este arquivo é processado por um compilador que gera o documento final (PDF). Essa abordagem garante que a consistência visual seja mantida automaticamente, independentemente do tamanho do projeto.

1.4 Estrutura de Projeto Profissional

Um projeto LaTeX com vários arquivos envolve mais do que escrever texto em um documento único. Ele exige organização em:

1. **Preâmbulo**: Configurações globais e carregamento de pacotes.

2. **Corpo do Texto**: Organizado de forma modular via `\input` ou `\include`.

3. **Gestão de Bibliografia**: Uso de arquivos `.bib` e motores como BibTeX ou Biber.

4. **Automação**: Uso de ferramentas como `latexmk` para gerenciar passagens sucessivas de compilação.

Este material foi formatado utilizando um template baseado no Carumã [1] (MIT License), adaptado à estrutura deste minicurso.

II
Compiladores, Fontes e Automação

O processo de compilação explica parte importante do comportamento do LaTeX. Referências cruzadas que aparecem como "??", sumários incompletos e mensagens de fonte não reconhecida geralmente estão ligados à interação entre arquivo-fonte, compilador, pacotes e arquivos auxiliares. Distinguir o editor do compilador e reconhecer o papel de cada componente do ecossistema reduz tentativas aleatórias de correção.

II.1 Ecossistema: Distribuição, Editor, Compilador, Classe e Pacote

O LaTeX não é um programa único, mas um conjunto de componentes interdependentes que interagem para gerar o documento final.

- **Distribuição TeX**: É o conjunto instalado de compiladores, pacotes, classes, fontes e programas auxiliares. As principais são o **TeX Live** e o **MiKTeX**. Em ambientes online, o **Overleaf** fornece uma distribuição gerenciada na nuvem.
- **Editor**: A interface de escrita. Exemplos incluem o **TeXstudio** (IDE dedicada), o **VS Code** (com LaTeX Workshop), o **Vim** (com VimTeX) e o **Emacs** (com **AUCTeX**). O editor apenas facilita a escrita e invoca o compilador.
- **Compilador**: O programa binário que processa o código. No ecossistema TeX, esses programas são frequentemente chamados de **motores** ou *engines*. Exemplos: `pdflatex`, `xelatex`, `lualatex`.
- **Classe (`.cls`)**: Define a estrutura e o layout base (ex.: `book`, `article`, `beamer`).
- **Pacote (`.sty`)**: Acrescenta funcionalidades específicas (ex.: `amsmath`, `graphicx`, `tikz`).

II.2 Fluxo de Compilação e Arquivos Auxiliares

Quando executamos o comando `pdflatex arquivo.tex`, o compilador realiza uma leitura sequencial do arquivo de entrada. Ele expande comandos, carrega pacotes, processa ambientes, monta caixas tipográficas e escreve o PDF. No entanto, nem toda informação necessária para o documento final está disponível no momento em que

o compilador passa por um trecho do arquivo. Elementos que dependem da estrutura global, como sumário, referências cruzadas, lista de figuras e citações bibliográficas, precisam ser registrados em arquivos auxiliares.

Essa é uma diferença essencial entre compilar um documento pequeno sem referências e compilar um projeto técnico. O arquivo `.tex` é a entrada principal, mas o estado do documento também passa por arquivos gerados durante a compilação. Em um projeto real, apagar esses arquivos auxiliares ou compilar apenas uma vez pode deixar o PDF em um estado intermediário.

Um documento mínimo sem referências já ilustra a separação entre fonte e saída:

```
\documentclass{article}
\begin{document}
Texto do documento.
\end{document}
```

Código-fonte II.1: Documento mínimo

Ao compilar esse arquivo, o resultado principal é o PDF, mas também aparecem arquivos auxiliares:

```
pdflatex minimo.tex
ls
```

Código-fonte II.2: Primeira compilação manual

II.2.1 Por que múltiplas compilações?

Considere uma referência cruzada para uma seção que aparece mais adiante no documento. Na primeira passagem, o compilador encon-

tra o comando `\label` e registra no arquivo `.aux` o número da seção e a página correspondente. Se uma chamada a `\ref` apareceu antes dessa informação estar consolidada, o PDF pode mostrar "??" ou um valor antigo. Na segunda passagem, o compilador lê o `.aux` produzido anteriormente e substitui a referência pelo valor correto.

O sumário segue a mesma lógica. Durante uma compilação, os comandos de seção escrevem entradas no arquivo `.toc`. Em uma passagem posterior, o LaTeX lê esse arquivo para montar o sumário. Por isso, um sumário recém-criado pode aparecer vazio na primeira compilação e correto apenas depois de uma nova passagem.

O exemplo abaixo contém uma referência cruzada simples. Na primeira compilação, a referência pode aparecer como "??". Na segunda, o número da seção já estará disponível no arquivo `.aux`.

```
\documentclass{article}
\begin{document}

Veja a Secao~\ref{sec:metodo}.

\section{Metodo}\label{sec:metodo}
Texto da secao.

\end{document}
```

Código-fonte II.3: Referência cruzada simples

O arquivo `.aux` produzido conterá uma entrada associando o label ao número da seção. O formato exato pode variar, mas a ideia é esta:

```
\newlabel{sec:metodo}{{1}{1}}
```

Código-fonte II.4: Informação registrada no arquivo auxiliar

II.2.2 Anatomia dos Arquivos Gerados

- `.aux`: Armazena labels, citações, contadores e outras informações usadas entre passagens.
- `.log`: Registro detalhado da compilação. Informa pacotes carregados, arquivos encontrados, avisos, erros e estatísticas finais.
- `.toc, .lof, .lot`: Arquivos lidos para construir sumário, lista de figuras e lista de tabelas.
- `.bbl`: Arquivo de bibliografia formatada, normalmente produzido por BibTeX ou Biber.
- `.out`: Arquivo auxiliar usado por pacotes como `hyperref` para estruturas de navegação do PDF.

Esses arquivos não são texto do documento, mas fazem parte do processo de construção. A automação existe justamente para decidir quando cada um deles precisa ser regenerado.

II.3 Compiladores: pdflatex, xelatex e lualatex

A escolha do compilador determina como o sistema lida com fontes e caracteres modernos.

1. `pdflatex`: O padrão estável. Usa o modelo tradicional de fontes do TeX (*Type 1*). É rápido e amplamente compatível, mas exige pacotes específicos para carregar diferentes fontes.

2. `xelatex`: Suporta nativamente Unicode (UTF-8) e permite o uso de fontes instaladas no sistema operacional (OTF/TTF) via pacote `fontspec`.

3. `lualatex`: Motor moderno baseado em LuaTeX, com suporte a Unicode, gestão de fontes via `fontspec` e possibilidade de extensão de funcionalidades através da linguagem Lua.

II.4 Fontes no pdfLaTeX

Este projeto utiliza o `pdflatex` por estabilidade e compatibilidade com ambientes comuns de ensino. Para produzir documentos em português nesse fluxo, configuramos três elementos:

- **Entrada (UTF-8)**: indica como o arquivo `.tex` codifica caracteres acentuados. Em versões recentes do LaTeX, UTF-8 é o padrão; em materiais legados, ainda é comum encontrar `\usepackage[utf8]{inputenc}`.

- **Saída (T1)**: a codificação de fonte `T1` (habilitada por `\usepackage[T1]{fontenc}`) é importante para línguas com acentos. Ela favorece hifenação, cópia de texto e busca no PDF.

- **Pacotes de Fontes**: Como o `pdflatex` não carrega arquivos `.ttf` diretamente, usamos pacotes como `ebgaramond` ou `newpxtext` que encapsulam as métricas necessárias.

Em um documento pequeno, a configuração típica fica no preâmbulo:

```
\documentclass{article}
\usepackage[T1]{fontenc}
\usepackage[utf8]{inputenc} % explicito em
    materiais legados
\usepackage[brazilian]{babel}
\usepackage{ebgaramond}

\begin{document}
Acao, hipotese, funcao, consequencia.
\end{document}
```

Código-fonte II.5: Configuração básica de fonte no pdfLaTeX

II.5 Fontes do sistema e arquivos de fonte

O modelo tradicional usado pelo `pdflatex` não trabalha diretamente com fontes instaladas no sistema operacional nem com arquivos `.ttf` ou `.otf` escolhidos pelo usuário no preâmbulo. Em

geral, uma família tipográfica precisa estar disponível como pacote LaTeX, com métricas, codificações e arquivos auxiliares preparados para o TeX. Por isso, no fluxo com `pdflatex`, é comum escolher fontes por pacotes:

```
\usepackage[T1]{fontenc}
\usepackage{newpxtext}
\usepackage{newpxmath}
```

Código-fonte II.6: Fonte por pacote no pdfLaTeX

Esse modelo é menos direto que selecionar um arquivo de fonte, mas tem uma vantagem prática em cursos e projetos reproduzíveis: se a distribuição TeX contém o pacote, o documento compila sem depender das fontes instaladas no sistema operacional do usuário.

Em `xelatex` e `lualatex`, a situação é diferente. Esses compiladores trabalham melhor com Unicode e podem usar o pacote `fontspec` para carregar fontes pelo nome com que aparecem no sistema:

```
\usepackage{fontspec}
\setmainfont{TeX Gyre Pagella}
\setsansfont{TeX Gyre Heros}
\setmonofont{DejaVu Sans Mono}
```

Código-fonte II.7: Fonte instalada no sistema com XeLaTeX ou LuaLaTeX

Também é possível carregar arquivos de fonte distribuídos junto com o projeto. Nesse caso, o documento deixa de depender do nome da fonte instalada no sistema, mas passa a depender da presença dos arquivos no caminho indicado:

```
\usepackage{fontspec}
\setmainfont[
    Path = fontes/,
    UprightFont = MinhaFonte-Regular.ttf,
    ItalicFont = MinhaFonte-Italic.ttf,
    BoldFont = MinhaFonte-Bold.ttf,
    BoldItalicFont = MinhaFonte-BoldItalic.ttf
]{MinhaFonte}
```

Código-fonte II.8: Arquivos locais de fonte com XeLaTeX ou LuaLaTeX

Esses exemplos não devem ser copiados para o preâmbulo real deste livro, pois o projeto compila com `pdflatex`. Eles são relevantes, contudo, para entender as escolhas possíveis em outros projetos. Usar fontes do sistema é conveniente em documentos pessoais e em ambientes controlados, mas reduz a portabilidade: outro computador pode não ter a mesma fonte, ou pode tê-la com nome interno diferente. Distribuir arquivos `.ttf` ou `.otf` junto com o projeto melhora a reprodutibilidade técnica, mas exige atenção à licença da fonte e à organização dos arquivos.

II.6 Microtipografia com microtype

O pacote `microtype` introduz ajustes finos na composição tipográfica. Ele não muda a estrutura lógica do documento nem substitui uma escolha adequada de fonte, mas melhora a aparência de parágra-

fos justificados ao permitir pequenas correções que seriam trabalhosas de fazer manualmente.

- **Protrusão**: permite que certos sinais de pontuação avancem levemente na margem, reduzindo irregularidades visuais.
- **Expansão**: aplica pequenas variações na largura dos caracteres para melhorar a justificação do parágrafo.

O efeito costuma ser discreto. O leitor não deve perceber "um recurso" específico, mas sim linhas com menos buracos, margens visualmente mais regulares e menos casos em que o algoritmo precisa escolher entre espaçamento ruim e hifenação excessiva. Em textos longos, essa diferença se acumula.

Há limites. `microtype` não corrige conteúdo mal estruturado, linhas com URLs enormes, tabelas largas ou comandos que impedem que o TeX encontre bons pontos de quebra. Nesses casos, a solução continua sendo revisar a estrutura do trecho, usar comandos apropriados para URLs, ajustar tabelas ou reformular o texto.

O uso usual do pacote é simples:

```
\usepackage[
    final,
    protrusion=true,
    expansion=true
]{microtype}
```

Código-fonte II.9: Ativando microtipografia

Em muitos projetos basta carregar `\usepackage{microtype}`. Opções explícitas são úteis quando se deseja documentar a intenção ou controlar o comportamento em uma classe específica.

II.7 Automação e Fluxos de Trabalho

A execução manual de comandos como `pdflatex`, `bibtex`, `pdflatex`, `pdflatex` é propensa a erros porque o ciclo correto depende do estado dos arquivos auxiliares. Um documento sem bibliografia pode precisar apenas de uma ou duas passagens. Um documento com citações, sumário e referências cruzadas pode exigir BibTeX ou Biber entre passagens de LaTeX. Se o usuário esquece uma etapa, o PDF pode compilar, mas sair com referências antigas, citações indefinidas ou sumário desatualizado.

`latexmk` automatiza essa decisão. Ele lê os arquivos gerados, observa mensagens do log e executa novamente os programas necessários até que o PDF esteja consistente ou até encontrar um erro que impeça a continuação. Em vez de memorizar uma sequência fixa, o usuário delega ao `latexmk` a tarefa de descobrir se precisa chamar `pdflatex`, BibTeX, Biber ou outra passagem de compilação.

Uma sequência manual típica para documento com BibTeX seria:

```
pdflatex artigo.tex
bibtex artigo
pdflatex artigo.tex
pdflatex artigo.tex
```

Código-fonte II.10: Ciclo manual com bibliografia

A primeira chamada de `pdflatex` lê o arquivo `artigo.tex`, monta a estrutura inicial do documento e grava arquivos auxiliares, como `artigo.aux`. Nesse arquivo ficam registradas informações

que ainda não necessariamente aparecem corretamente no primeiro PDF: chaves de citação, rótulos, entradas de sumário e referências cruzadas.

Em seguida, `bibtex artigo` não recebe o arquivo `artigo.tex`, mas sim o nome base do documento. O BibTeX lê `artigo.aux`, identifica quais obras foram citadas, consulta os arquivos `.bib` indicados no projeto e gera um arquivo `artigo.bbl`. Esse arquivo contém a bibliografia já formatada segundo o estilo bibliográfico em uso.

A segunda chamada de `pdflatex` incorpora o conteúdo de `artigo.bbl` ao documento e começa a resolver as citações no texto. Ainda assim, algumas referências internas podem mudar de página ou de numeração depois que a bibliografia entra no PDF. Por isso, a terceira chamada de `pdflatex` serve para estabilizar referências cruzadas, sumário, citações e demais informações que dependem dos arquivos auxiliares.

Com `latexmk`, essa sequência passa a ser delegada à ferramenta:

```
latexmk -pdf -pdflatex="pdflatex -interaction
    nonstopmode" artigo.tex
```

Código-fonte II.11: Ciclo automático com latexmk

- `latexmk`: Monitora dependências e executa o ciclo de compilação necessário para atualizar o PDF.
- **Editor local**: Programas como TeXstudio ou extensões de VS Code podem chamar `pdflatex` ou `latexmk` por botões e atalhos.

- **Ambiente online**: Plataformas como Overleaf executam o ciclo de compilação por trás da interface, mas o log continua sendo a fonte principal de diagnóstico.

II.8 Erros e Diagnóstico: Lendo o Log

A primeira regra da depuração em LaTeX é: **ignore os últimos erros e foque no primeiro**. Frequentemente, um único erro de sintaxe gera uma cascata de falhas subsequentes. Depois que o compilador perde o ponto exato da estrutura do documento, ele pode interpretar linhas seguintes de forma incorreta e emitir mensagens que são consequência, não causa.

No terminal e no arquivo `.log`, erros aparecem com uma linha iniciada por `!`. Logo abaixo, o LaTeX costuma mostrar o contexto e, em muitos casos, o número da linha do arquivo `.tex`. O procedimento básico é: localizar o primeiro `!`, identificar o arquivo ativo, verificar a linha indicada e ler também algumas linhas antes dela. Muitos erros são causados por um comando aberto anteriormente, uma chave não fechada ou um ambiente iniciado sem o fechamento correspondente.

Também é importante distinguir erro de aviso. Um erro interrompe ou compromete a compilação; um aviso informa uma situação que pode ou não exigir ação. Mensagens como `Label(s) may have changed` indicam que uma nova compilação pode resolver

o problema. Já `Overfull \hbox` aponta uma decisão tipográfica ruim, mas normalmente não impede a geração do PDF.

Um erro de comando inexistente costuma aparecer assim:

```
! Undefined control sequence.
l.12 \seciton
                {Introducao}
```

Código-fonte II.12: Exemplo de erro no log

O trecho mostra três informações: o tipo do erro, a linha aproximada e o ponto em que o compilador deixou de reconhecer o comando. Nesse caso, a causa provável é um erro de digitação em `\section`.

`Undefined control sequence` Comando inexistente ou pacote não carregado. Verifique se digitou `\section` em vez de `\secton`.

`File not found` O compilador não encontrou uma imagem ou arquivo incluído. Verifique caminhos e extensões.

`Missing $ inserted` Ocorreu o uso de um símbolo matemático (como _) fora do modo matemático.

`Overfull \hbox` O texto excedeu a margem lateral. Ajuste a hifenação ou reformule o parágrafo.

II.9 Consulta de Documentação

A documentação do ecossistema TeX costuma ser instalada junto com a distribuição. Isso é uma diferença importante em relação a muitos tutoriais encontrados na web: quando um pacote está instalado localmente, em geral o seu manual também está. O comando `texdoc` é a interface mais simples para acessar esses manuais sem precisar procurar arquivos PDF manualmente dentro da árvore da distribuição.

Na prática, `texdoc` recebe o nome de um pacote, classe ou ferramenta e tenta abrir a documentação correspondente. Para pacotes muito usados, o resultado costuma ser direto:

```
texdoc amsmath
texdoc graphicx
texdoc beamer
```

Esses comandos abrem, respectivamente, a documentação do `amsmath`, o guia de gráficos do LaTeX e o manual do Beamer. Também é possível consultar ferramentas de linha de comando e classes:

```
texdoc latexmk
texdoc book
texdoc listings
```

Nem toda busca retorna um único resultado. Alguns nomes são ambíguos, e o `texdoc` pode apresentar uma lista numerada para escolha. Isso não é erro: significa apenas que a distribuição contém mais de um documento relacionado ao termo pesquisado. Nesses ca-

sos, vale observar o nome do arquivo, o pacote associado e a descrição curta exibida pelo próprio comando.

Ao ler um manual, a estratégia não deve ser tentar absorver tudo de uma vez. Manuais de pacotes como `beamer`, `pgf`/TikZ ou `biblatex` podem ter centenas de páginas. O uso mais eficiente é localizar a seção relevante para a tarefa atual: opções de pacote, sintaxe de um ambiente, exemplos mínimos, avisos de compatibilidade e limitações conhecidas. O índice, a busca textual no PDF e os exemplos oficiais costumam ser mais úteis do que uma leitura linear.

Quando a documentação local não existe, está desatualizada ou não corresponde à versão usada em outro ambiente, o próximo ponto de consulta é o **CTAN** (`https://ctan.org`). O CTAN reúne pacotes, documentação, histórico de versões e links para os arquivos distribuídos. Para problemas de compilação, entretanto, é importante comparar a documentação consultada com a versão efetivamente instalada, pois uma opção disponível em uma versão recente pode não existir em uma distribuição TeX mais antiga.

Há também uma interface online inspirada no `texdoc`: `https://texdoc.org`. Ela permite buscar documentação de pacotes pelo navegador, de modo semelhante a uma consulta local com `texdoc -l`. Esse recurso é útil quando o aluno está em uma máquina sem a documentação instalada, usando um ambiente online, ou apenas quer consultar rapidamente um manual sem abrir o terminal. Ainda assim, a mesma cautela vale: o documento encontrado online pode corresponder a uma versão diferente da instalada localmente.

Uma forma prática de organizar as consultas é distinguir o papel de cada fonte. Use `texdoc` quando estiver trabalhando em uma instala-

ção local e quiser a documentação que acompanha a sua distribuição. Use `https://texdoc.org` quando precisar de uma busca rápida pelo navegador. Use o CTAN quando quiser verificar a página oficial do pacote, arquivos distribuídos, histórico e versão mais recente.

Como regra de trabalho, prefira a documentação oficial do pacote antes de recorrer a respostas soltas de fóruns ou tutoriais. Materiais externos podem ser úteis, especialmente para exemplos, mas frequentemente omitem versões, opções carregadas no preâmbulo ou restrições de compilador. Em um projeto que precisa ser reproduzível, esses detalhes importam.

III
Documentos técnicos e escrita matemática

Depois de entender o ciclo de compilação, os arquivos auxiliares e o papel dos pacotes, o próximo passo é usar essa infraestrutura para escrever documentos técnicos que continuam legíveis quando crescem. Um texto matemático pequeno tolera improvisos: uma equação isolada, um comando escrito diretamente no corpo do texto, uma referência feita à mão. Em um artigo, uma dissertação, uma lista extensa ou uma apostila, esses improvisos passam a gerar inconsistências.

Este capítulo trata de três problemas recorrentes: organizar o documento em partes, compor matemática com os ambientes apropriados e criar comandos semânticos para a notação usada no texto. O objetivo não é decorar todos os comandos dos pacotes da American Mathematical Society, mas entender quais decisões eles permitem tomar.

III.1 Estrutura de um documento técnico

Um documento técnico tende a crescer em camadas. O preâmbulo reúne decisões globais: classe, pacotes, fontes, idioma, estilo de referências e comandos próprios. O corpo do documento contém o texto. Arquivos auxiliares são gerados pelo compilador e registram informações necessárias entre passagens, como labels, citações e sumário.

Misturar todas essas camadas no mesmo arquivo funciona em exemplos mínimos, mas dificulta manutenção. Uma forma simples de modularização é manter um arquivo principal e carregar capítulos ou seções com `\input`:

```
\documentclass{book}

\usepackage[T1]{fontenc}
\usepackage[utf8]{inputenc}
\usepackage[brazilian]{babel}
\usepackage{mathtools}
\usepackage{amssymb}

\begin{document}

\input{introducao}
\input{preliminares}
\input{resultados}

```

```
\end{document}
```

Código-fonte III.1: Arquivo principal com capítulos separados

O comando `\input` copia o conteúdo do arquivo indicado naquele ponto da compilação. Para o leitor e para o compilador, o resultado é como se o conteúdo estivesse escrito no arquivo principal. A vantagem é organizacional: cada capítulo pode ser editado separadamente, e o arquivo principal continua pequeno o bastante para mostrar a estrutura geral do projeto.

Em documentos técnicos, nomes previsíveis ajudam. Arquivos como `preliminares.tex`, `resultados.tex` e `apendice-demonstracoes.tex` comunicam melhor o papel do arquivo do que nomes genéricos como `parte1.tex`. O mesmo vale para labels: `sec:preliminares`, `eq:energia` e `fig:dominio` são mais úteis do que `label1`.

III.2 A pilha AMS

O pacote `amsmath` é a base moderna para escrita matemática em LaTeX. Ele fornece ambientes de equações, comandos para operadores, controle de numeração, matrizes, casos, setas extensíveis, frações e vários ajustes tipográficos. O pacote `amssymb` adiciona símbolos matemáticos comuns em textos avançados. O pacote `mathtools` estende o `amsmath`: carrega `amsmath` automaticamente e oferece ferramentas para delimitadores, versões refinadas de ambientes e comandos úteis para ajustes finos.

Uma configuração comum para documentos com bastante matemática é:

```
\usepackage{mathtools} % carrega amsmath
\usepackage{amssymb}
```

Código-fonte III.2: Pacotes para escrita matemática

Também é comum encontrar:

```
\usepackage{amsmath}
\usepackage{amssymb}
```

Código-fonte III.3: Carregamento explícito de amsmath

As duas escolhas são aceitáveis. Quando o documento usa recursos de `mathtools`, carregar apenas `\usepackage{mathtools}` evita redundância. Quando o documento usa apenas recursos clássicos da AMS, `\usepackage{amsmath}` deixa a dependência explícita.

III.3 Ambientes de equações

Um erro comum é tratar todos os displays matemáticos como se fossem iguais. A escolha do ambiente comunica a estrutura da fórmula: uma equação isolada, uma equação longa quebrada em linhas, um grupo de equações independentes ou um cálculo alinhado por relações.

Para uma equação única numerada, use `equation`. Para uma equação única sem número, use `\[...\]` ou `equation*`:

```
\begin{equation}\label{eq:energia}
E = mc^2
\end{equation}
```

```

\[
E = mc^2
\]
```

Código-fonte III.4: Equação isolada

Quando uma equação é longa, mas ainda representa uma única igualdade ou identidade, `split` dentro de `equation` permite quebrar linhas mantendo um único número:

```
\begin{equation}\label{eq:funcional}
\begin{split}
J(u)
  &= \int_\Omega |\nabla u(x)|^2\,dx \\
  &\quad + \int_\Omega V(x)|u(x)|^2\,dx
        - \int_\Omega F(x,u(x))\,dx .
\end{split}
\end{equation}
```

Código-fonte III.5: Equação longa com split

O ponto de alinhamento vem antes do símbolo relacional. Em `align`, por exemplo, escreve-se `&=`, não `=&`. Isso preserva o espaçamento matemático esperado ao redor da relação.

```
\begin{align}
\|u+v\|^2
  &= \langle u+v, u+v\rangle \\
  &= \|u\|^2 + 2\langle u,v\rangle + \|v\|^2 .
\end{align}
```

Código-fonte III.6: Cálculo alinhado

Quando há várias equações sem alinhamento comum, `gather` é mais adequado que `align`. Quando há uma equação longa sem

ponto natural de alinhamento, `multline` pode ser melhor que forçar um alinhamento artificial.

```
\begin{gather}
A+B+C = 0,\\
\int_\Omega f(x)\,dx = 1.
\end{gather}
```

Código-fonte III.7: Equações sem alinhamento comum

O ambiente `eqnarray` deve ser evitado em documentos novos. Ele produz espaçamento inadequado ao redor de relações e lida mal com numeração em equações longas. Para documentos já existentes, a substituição por `align` ou por `equation` com `split` costuma ser uma melhoria objetiva.

III.4 Numeração e referências

Referências para equações devem usar labels, não números digitados manualmente. O comando `\eqref`, fornecido por `amsmath`, imprime a referência com parênteses:

```
\begin{equation}\label{eq:identidade}
(a+b)^2 = a^2 + 2ab + b^2
\end{equation}

Pela identidade \eqref{eq:identidade}, obtemos
    ...
```

Código-fonte III.8: Referência a equação

Em ambientes com várias linhas, nem toda linha precisa ser numerada. Use `\notag` quando a linha não deve receber número:

```
\begin{align}
f(x) &= x^2 + 2x + 1 \notag\\
     &= (x+1)^2. \label{eq:quadrado}
\end{align}
```

Código-fonte III.9: Controle de numeração por linha

Quando um grupo de equações deve compartilhar uma numeração principal, com letras subordinadas, use `subequations`:

```
\begin{subequations}\label{eq:sistema}
\begin{align}
-\Delta u &= f && \text{em } \Omega,\\
u &= 0 && \text{em } \partial\Omega.
\end{align}
\end{subequations}
```

Código-fonte III.10: Numeração subordinada

Esse padrão é útil quando o texto precisa referir-se ao sistema inteiro e também a cada equação individual.

III.5 Texto dentro de matemática

Texto dentro de uma fórmula não deve ser improvisado com `\mathrm` quando a intenção é escrever uma palavra ou uma frase. O comando `\text` preserva o tamanho adequado em índices e mantém o texto como texto:

```
\[
P_{r-j} =
\begin{cases}
```

```
0, & \text{se } r-j \text{ e impar},\\
1, & \text{se } r-j \text{ e par}.
\end{cases}
\]
```

Código-fonte III.11: Texto em uma fórmula

Em cálculos alinhados, justificativas curtas podem aparecer em uma coluna separada:

```
\begin{align}
x &= y_1 - y_2 + y_3 && \text{por \eqref{eq:
    recorrencia}}\\
  &= y^\prime \circ y^* && \text{pela definicao
    de composicao}.
\end{align}
```

Código-fonte III.12: Justificativas em align

Quando uma frase precisa interromper um cálculo mantendo o alinhamento, `\intertext` é apropriado. Para uma interrupção muito curta, `mathtools` oferece `\shortintertext`, que usa menos espaço vertical.

```
\begin{align}
A_1 &= N_0(\lambda;\Omega') - \phi(\lambda;\
    Omega'),\\
A_2 &= \phi(\lambda;\Omega') - \phi(\lambda;\
    Omega),
\shortintertext{e portanto}
A_3 &= N(\lambda;\omega).
\end{align}
```

Código-fonte III.13: Interrupção preservando alinhamento

III.6 Operadores matemáticos

Funções como `\sin`, `\log` e `\lim` são compostas em fonte romana e recebem espaçamento próprio. O mesmo tratamento deve ser dado a operadores definidos pelo autor. Escrever `rank A`, `\mathrm{rank} A` ou `Rank A` diretamente no corpo do documento resolve apenas o caso local; declarar o operador resolve a notação do documento inteiro.

```
\DeclareMathOperator{\rank}{rank}
\DeclareMathOperator{\tr}{tr}
\DeclareMathOperator{\id}{id}

\[
\rank A + \rank B \geq \rank(A+B),
\qquad
\tr(AB) = \tr(BA).
\]
```

Código-fonte III.14: Declaração de operadores

Quando o operador deve receber limites abaixo e acima em modo display, usa-se a forma estrelada:

```
\DeclareMathOperator*{\argmin}{arg\,min}
\DeclareMathOperator*{\esssup}{ess\,sup}

\[
u = \argmin_{v\in V} J(v),
\qquad
\esssup_{x\in\Omega} |f(x)| < \infty .
\]
```

Código-fonte III.15: Operador com limites

Essa prática também torna o código mais semântico. O comando `\argmin` diz o que está sendo escrito; uma sequência manual de letras, espaços e fontes apenas tenta reproduzir uma aparência.

III.7 Macros semânticas

Macros próprias são uma das ferramentas mais importantes em documentos matemáticos. Elas reduzem repetição, evitam inconsistências e separam a notação do modo como ela é desenhada. Se uma notação aparece dezenas de vezes, ela deve ter um nome.

```
\newcommand{\R}{\mathbb{R}}
\newcommand{\N}{\mathbb{N}}
\newcommand{\eps}{\varepsilon}
\newcommand{\dd}{\,d}

\[
\int_{\R^n} f(x)\dd x < \eps.
\]
```

Código-fonte III.16: Macros para notação recorrente

O nome da macro deve representar o significado pretendido. Em vez de criar apenas abreviações visuais, como `\bbR`, prefira comandos que revelem o papel da notação no texto quando isso for possível. Em textos longos, comandos como `\norm`, `\abs`, `\inner`, `\Prob` e `\E` ajudam a manter consistência.

III.8 Definição de comandos

Criar comandos próprios é parte normal da escrita de documentos técnicos em LaTeX. O ponto importante é distinguir comandos que representam notação matemática de comandos que apenas encurtam digitação. Uma macro bem escolhida torna o código-fonte mais legível e permite alterar a notação de forma controlada.

A forma usual para criar comandos em documentos LaTeX é `\newcommand`. Ela verifica se o comando já existe e emite erro se houver conflito. Esse comportamento é desejável: se um nome já foi definido por uma classe, por um pacote ou por outra parte do projeto, sobrescrevê-lo silenciosamente pode produzir efeitos difíceis de diagnosticar.

```
\newcommand{\R}{\mathbb{R}}
\newcommand{\eps}{\varepsilon}
\newcommand{\norma}[1]{\lVert#1\rVert}

\[
u \in \R^n,
\qquad
\norma{u} < \eps.
\]
```

Código-fonte III.17: Definição segura de comandos

Comandos com argumentos usam marcadores como `#1`, `#2` e assim por diante. No exemplo, `\norma{u}` substitui `#1` por `u`. Para notações mais estruturadas, vale escolher nomes que descrevam a operação:

```
\newcommand{\restricao}[2]{#1\vert_{#2}}
```

```
\newcommand{\derivada}[2]{\frac{d #1}{d #2}}

\[
\restricao{f}{A},
\qquad
\derivada{y}{x}.
\]
```

Código-fonte III.18: Comandos com argumentos

Quando a intenção é redefinir um comando existente, use `\renewcommand`. Isso deixa explícito que a redefinição é deliberada. Quando o comando deve ser criado apenas se ainda não existir, use `\providecommand`. Essa última forma é comum em arquivos compartilhados entre projetos, porque permite definir uma macro padrão sem sobrescrever uma definição feita pelo documento principal.

```
\renewcommand{\epsilon}{\varepsilon}
\providecommand{\dd}{\,d}
```

Código-fonte III.19: Redefinição e definição condicional

Em código TeX mais antigo ou em implementações internas de pacotes, aparece frequentemente o primitivo `\def`. Ele define comandos de forma mais direta e não faz as verificações de segurança de `\newcommand`. Por isso, em material de curso e em documentos de usuário, `\newcommand` e `\renewcommand` são escolhas melhores. `\def` é útil para entender código de baixo nível, mas não deve ser usado como padrão em documentos técnicos comuns.

```
% Forma primitiva: comum em codigo de pacote,
% mas menos segura para documentos de usuario.
```

```
\def\R{\mathbb{R}}
```

Código-fonte III.20: Definição primitiva em TeX

III.9 Cabeçalhos e rodapés

Cabeçalhos e rodapés fazem parte da estrutura editorial do documento. Eles ajudam o leitor a se localizar, especialmente em textos longos impressos ou em PDFs consultados fora do editor. Em um livro, é comum colocar o número da página no rodapé e informações como título do livro, autor ou capítulo no cabeçalho. Em um artigo, a escolha pode ser mais discreta ou até inexistente, dependendo da classe e das normas de submissão.

O pacote mais usado para esse controle é `fancyhdr`. Ele permite limpar o estilo padrão da página, definir campos de cabeçalho e rodapé e controlar a linha horizontal do cabeçalho:

```
\usepackage{fancyhdr}

\pagestyle{fancy}
\fancyhf{} % limpa cabecalho e rodape

\fancyhead[LE]{\nouppercase{\leftmark}}
\fancyhead[RO]{\nouppercase{\rightmark}}
\fancyfoot[C]{\thepage}

\renewcommand{\headrulewidth}{0.4pt}
\renewcommand{\footrulewidth}{0pt}
```

Código-fonte III.21: Cabeçalho e rodapé com fancyhdr

As letras usadas por `fancyhdr` indicam a posição: `L`, `C` e `R` significam esquerda, centro e direita; `E` e `O` significam páginas pares e ímpares. Em documentos frente e verso, essa distinção é relevante porque a margem interna muda de lado. Em documentos simples, muitas vezes basta usar `\fancyfoot[C]{\thepage}`.

Também é possível definir um estilo próprio para páginas especiais, como a primeira página de um capítulo. Isso evita que uma página de abertura receba o mesmo cabeçalho de uma página comum:

```
\fancypagestyle{plain}{%
  \fancyhf{}
  \fancyfoot[C]{\thepage}
  \renewcommand{\headrulewidth}{0pt}
}
```

Código-fonte III.22: Estilo para páginas especiais

Do ponto de vista técnico, cabeçalho e rodapé não devem carregar informação demais. Se o cabeçalho fica mais saliente que o texto, ele deixa de orientar e passa a competir com o conteúdo. Em documentos matemáticos, a prioridade continua sendo legibilidade: número de página claro, marcas de capítulo ou seção quando úteis, e pouco ruído visual.

III.10 Delimitadores e notação pareada

O uso automático de `\left` e `\right` parece conveniente, mas nem sempre produz o melhor resultado. Esses comandos escolhem o tamanho mecanicamente e podem criar delimitadores grandes demais em

expressões pequenas. Em muitos casos, tamanhos explícitos como `\bigl`, `\bigr`, `\Bigl` e `\Bigr` produzem resultado mais controlado.

Para valor absoluto e norma, `amsmath` recomenda comandos distintos para barras esquerda e direita:

```
\newcommand{\abs}[1]{\lvert#1\rvert}
\newcommand{\norm}[1]{\lVert#1\rVert}

\[
\abs{x-y} \leq \norm{u-v}.
\]
```

Código-fonte III.23: Delimitadores com amsmath

O `mathtools` generaliza essa ideia com `\DeclarePairedDelimiter`:

```
\DeclarePairedDelimiter{\abs}{\lvert}{\rvert}
\DeclarePairedDelimiter{\norm}{\lVert}{\rVert}

\[
\abs{x}
\qquad
\abs*{\frac{x}{1+x^2}}
\qquad
\norm[\Big]{\sum_{k=1}^n v_k}
\]
```

Código-fonte III.24: Delimitadores com mathtools

A versão sem estrela usa o tamanho natural. A versão estrelada usa delimitadores automáticos. A versão com argumento opcional per-

mite escolher explicitamente o tamanho. Essa combinação preserva uma interface semântica e ainda permite ajuste tipográfico local.

Para notações com mais estrutura, como produto interno ou conjuntos por compreensão, `mathtools` fornece `\DeclarePairedDelimiterX`:

```
\DeclarePairedDelimiterX{\inner}[2]{\langle}{\
    rangle}{#1,#2}

\providecommand{\given}{}
\newcommand{\SetSymbol}[1][]{%
  \nonscript\:#1\vert\allowbreak\nonscript\:\
    mathopen{}}
\DeclarePairedDelimiterX{\Set}[1]{\{}{\}}{%
  \renewcommand{\given}{\SetSymbol[\delimsize
    ]}#1}

\[
\inner{u}{v}
\qquad
\Set*{x\in X \given f(x)>0}
\]
```

Código-fonte III.25: Produto interno e conjunto por compreensão

Esse exemplo é mais avançado, mas ilustra uma ideia central: a macro deve expressar a notação matemática, não apenas economizar digitação.

III.11 Matrizes, casos e pontos

O `amsmath` oferece ambientes de matrizes com delimitadores prontos: `pmatrix`, `bmatrix`, `vmatrix`, `Vmatrix` e outros. Para matrizes pequenas dentro de texto, `smallmatrix` evita uma construção grande demais para a linha.

```
\[
A =
\begin{pmatrix}
1 & 0\\
0 & 1
\end{pmatrix},
\qquad
\det A =
\begin{vmatrix}
a & b\\
c & d
\end{vmatrix}.
\]
```

Código-fonte III.26: Matrizes com amsmath

O `mathtools` também fornece variantes úteis, como `dcases`, em que o conteúdo dos casos é composto em estilo de display:

```
\[
f(x) =
\begin{dcases}
\frac{\sin x}{x}, & x\neq 0,\\
1, & x = 0.
\end{dcases}
```

```
\]
```

Código-fonte III.27: Casos em estilo de display

Para reticências, comandos semânticos como `\dotsc`, `\dotsb`, `\dotsm` e `\dotsi` comunicam o contexto: vírgulas, operadores binários, multiplicação ou integrais. Essa distinção permite que o pacote escolha a forma tipográfica apropriada.

```
\[
a_1, a_2, \dotsc, a_n
\qquad
a_1 + a_2 + \dotsb + a_n
\qquad
a_1 a_2 \dotsm a_n
\]
```

Código-fonte III.28: Reticências semânticas

III.12 Um exemplo de refatoração

O trecho abaixo compila, mas mistura decisões tipográficas, notação e estrutura:

```
\begin{eqnarray}
||u+v||^2 &=& <u+v,u+v>\\
&=& ||u||^2 + 2<u,v> + ||v||^2.
\end{eqnarray}
```

Código-fonte III.29: Trecho matemático frágil

Uma versão mais robusta usa ambiente adequado, macros semânticas e delimitadores declarados:

```
\DeclarePairedDelimiter{\norm}{\lVert}{\rVert}
\DeclarePairedDelimiterX{\inner}[2]{\langle}{\
    rangle}{#1,#2}

\begin{align}
\norm{u+v}^2
  &= \inner{u+v}{u+v}\\
  &= \norm{u}^2 + 2\inner{u}{v} + \norm{v}^2.
\end{align}
```

Código-fonte III.30: Trecho refatorado

A diferença não é apenas visual. Na segunda versão, o ambiente expressa que há um cálculo alinhado; os comandos `\norm` e `\inner` expressam a notação; a troca de estilo dos delimitadores pode ser feita em um único ponto do preâmbulo.

III.13 Critérios práticos

Ao escrever matemática em um documento técnico, algumas perguntas ajudam a escolher a estrutura:

- A fórmula é uma equação isolada ou parte de um cálculo?
- Cada linha precisa de número?
- Há um ponto matemático natural de alinhamento?
- A notação aparece muitas vezes no documento?

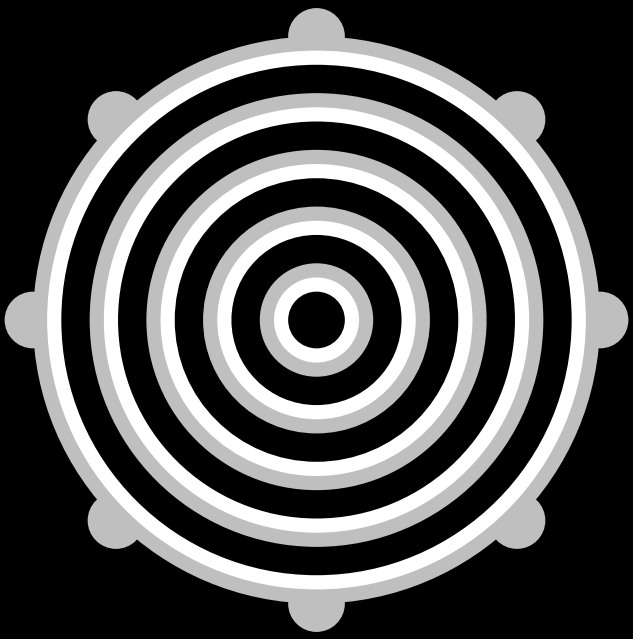
- O delimitador deve ter tamanho automático ou tamanho controlado?
- O texto dentro da fórmula é uma palavra, uma justificativa ou parte da notação?
- A informação repetida em cabeçalhos e rodapés ajuda a navegação ou apenas adiciona ruído?

Essas perguntas evitam uma escrita matemática puramente visual. Em LaTeX, a qualidade do resultado depende de escolher estruturas que correspondem ao papel matemático do trecho.

Referências Bibliográficas

[1] Janderson Gomes. Caraumã, 2021. Template disponível em https://github.com/jancarauma/latex-ebook-template e https://www.overleaf.com/latex/templates/carauma/pjksmbfyrnkr.

LaTeX para gente grande



LEIII